

Yes. We can park Incident 01 exactly where it is and now treat **Incident 02 as a complete playable operation**.

The important thing is that Incident 02 must **not solve everything**. Its job is to make the learner discover why encryption is useful and then discover the next problem created by symmetric encryption:

Protecting the information is possible — but Bob also needs the same secret key. How do we get that secret to him safely?

That unresolved problem is what hands us into Incident 03.

Incident 02 — Protect the Information

1. Core purpose

Incident 01 ended with:

PQ-001 INITIAL ASSESSMENT

Availability	AFFECTED
Integrity	AFFECTED
Confidentiality	IN QUESTION
Identity Activity	SUSPICIOUS
Actor	UNKNOWN
Root Cause	UNKNOWN

and the operational conclusion:

Information cannot currently be trusted to move safely between authorised parties.

Alice still needs to communicate with Bob.

So Incident 02 begins with a practical question:

How can Alice send information without exposing it in readable form while it is moving?

The learner is not told:

“Now learn symmetric cryptography.”

The situation creates the need for it.

2. Starting state

The trainee returns to Central Operations after completing the PQ-001 assessment.

A new clearance notification appears:

CLEARANCE UPDATED

CRYPTOGRAPHY LAB

ACCESS GRANTED ✓

But only the **basic symmetric-security workbench** is available.

The learner has:

KNOWN

Information may be lost or altered in transit.

Confidentiality remains uncertain.

Alice and Bob are authorised parties.

NOT YET KNOWN

What encryption is operationally.

How keys work.

How Bob gets the key.

3. The object carried forward

We should deliberately reuse:

03_ACCESS_SCHEDULE.XLSX

That gives continuity.

Alice still owns the source.

She needs Bob to receive the information.

The learner already recognises the file from the Prologue and Incident 01.

That makes Incident 02 feel like:

We are now trying to solve the actual problem we just investigated.

4. Incident 02 overall structure

We previously established roughly **14 role-play scenes**, and I would preserve that structure:

```
01 Cryptography Lab unlocks
02 Inspect the readable information
03 Ordinary transfer is blocked
04 Create shared secret Key A7
05 Build the protection process
06 Convert plaintext → ciphertext
07 Send protected information
08 Bob receives ciphertext
09 Bob cannot open it
10 Discover Bob needs A7
11 Test normal key transfer
12 Discover exposure problem
13 Test physical secure transfer
14 Approve a controlled key-transfer method
```

And then stop.

Incident 02 should end with:

```
FILE PROTECTED ✓
KEY CREATED ✓
TRANSFER METHOD SELECTED ✓
BOB STILL DOES NOT YET HAVE A7
```

Incident 03 performs the actual identity verification and key release.

5. ACT A — The Need for Protection

Scene 1 — Enter the Cryptography Lab

The learner physically walks from Central Operations into the newly unlocked Cryptography Lab.

This room should feel connected to HQ, but different from Training & Foundations.

The workstation shows:

```
PQ-001 RESPONSE

OBJECTIVE:
Protect information before transfer
```

Alice is present with:

```
03_ACCESS_SCHEDULE.XLSX
```

Bob is remote.

Player action

Open the file at the cryptography workstation.

Scene 2 — Inspect the Readable Information

The learner sees normal readable content.

We introduce the term only after they see it:

PLAINTEXT

The system might highlight:

```
Readable by a person  
Readable by any application that opens it
```

The learning message is simple:

Plaintext is information in a readable form.

No history of cryptography.

No Caesar cipher lecture.

No maths.

Scene 3 — Try the Normal Transfer

The learner attempts the same style of send they previously used.

```
03_ACCESS_SCHEDULE.XLSX  
↓  
SEND TO BOB
```

The system blocks it:

```
TRANSFER BLOCKED
```

And perhaps the lab gives the operational reason:

PQ-001 demonstrated that
the current transfer process
cannot yet be trusted.

This is important.

The learner sees:

We are not encrypting because the curriculum says so. We are encrypting because the current security state requires it.

6. ACT B — Build Symmetric Protection

Scene 4 — Create a Shared Secret

The workbench introduces a new object:

SHARED SECRET

The learner creates:

KEY A7

A7 becomes a persistent security object from this point onward.

Initially:

ALICE

A7 ✓

BOB

A7 ✗

The learner doesn't necessarily need the terminology “symmetric” immediately.

First let them use the same-key relationship.

Scene 5 — Build the Protection Process

The player gets three functional objects:

PLAINTEXT
KEY A7
PROTECT

They physically arrange:

PLAINTEXT
+
KEY A7
↓
ENCRYPT

The system then asks for an output.

Correct:

CIPHERTEXT

Only now introduce:

Encryption transforms plaintext into ciphertext using a key.

Scene 6 — Run the Process

The learner activates the workbench.

Visually:

03_ACCESS_SCHEDULE.XLSX

PLAINTEXT
+
A7
↓
ENCRYPT
↓
CIPHERTEXT

The output should look unreadable to the learner.

The game should make the transformation visually obvious.

7. What should the learner understand here?

Not:

“Encryption means the file is magically safe.”

Only:

BEFORE

Readable information

AFTER

Protected representation

And:

Someone encountering the ciphertext should not simply see the original information.

We keep the claim appropriately limited.

8. ACT C — Send the Protected Information

Scene 7 — Send Ciphertext to Bob

The learner now performs:

CIPHERTEXT

↓

SEND

↓

BOB

This time Phantom Q allows it.

The transfer completes.

PROTECTED FILE

DELIVERED ✓

This should feel like progress.

For a moment, the player may believe the problem is solved.

Good.

Scene 8 — Bob Receives It

Bob's remote workstation shows:

```
03_ACCESS_SCHEDULE.XLSX
PROTECTED PACKAGE

RECEIVED ✓
```

But when he attempts to open it:

```
CONTENT
UNAVAILABLE
```

or:

```
DECRYPTION KEY REQUIRED
```

Bob says something operational, such as:

“I have the file, but I can't read it.”

This is the next problem emerging naturally.

Scene 9 — Why Can't Bob Read It?

The player compares:

```
ALICE

Ciphertext ✓
Key A7      ✓
```

against:

```
BOB

Ciphertext ✓
Key A7      ✗
```

The workbench then demonstrates:

```
CIPHERTEXT
+
?
```

↓
DECRYPT

↓
CANNOT COMPLETE

The learner should infer:

Bob needs the same key.

Scene 10 — Discover the Symmetric Relationship

Now the term can appear:

SYMMETRIC SECURITY

Visually:

ALICE
KEY A7

↑

BOB
KEY A7

The rule:

The same shared secret is used by both authorised parties.

And:

ALICE KEY
A7

BOB REQUIRED KEY
A7

This is the central learning objective.

9. But don't immediately give Bob A7

This is where Incident 02 becomes more than:

click Encrypt → click Decrypt.

The learner now has a new problem:

How does Bob receive A7?

The file can safely travel as ciphertext.

But A7 itself is secret.

That distinction drives the rest of the incident.

10. ACT D — Explore Key Distribution

Scene 11 — Try the Normal Communication Route

The learner's first obvious action may be:

```
KEY A7
↓
SEND TO BOB
```

using the same normal communication channel.

Before blocking it, the game can show what that means.

Something like:

```
ALICE
  |
  | A7
  ▼
COMMUNICATION CHANNEL
  |
  ▼
BOB
```

The workbench marks:

```
SHARED SECRET
EXPOSED DURING DELIVERY ⚠
```

The idea:

If the secret itself must travel through the same path we are worried about, we have created another security problem.

This is the key-distribution problem.

Scene 12 — Why Sending the Key Normally Is Weak

We should make the learner discover this, rather than lecture.

The simulation can display:

```
CIPHERTEXT  
OBSERVED  
→ still protected  
  
KEY A7  
OBSERVED  
→ protection may be defeated
```

That's enough.

No Eve intercepting.

No unknown attacker shown.

Just a channel-risk simulation.

Important Eve rule

Eve can be present as:

```
SECURITY SUPPORT  
CHANNEL TEST / RISK REVIEW
```

She might say:

“If the secret can be obtained during delivery, the protected message may no longer remain protected.”

But she is **not shown stealing A7**.

This remains an authorised analysis environment.

11. ACT E — Consider Physical Delivery

Scene 13 — Secure Physical Token

Now present another legitimate option.

For example:

A7

↓

SECURE PHYSICAL TOKEN

↓

DELIVER TO BOB

This gives:

SECRET EXPOSURE

LOWER ✓

REMOTE DELIVERY

NO ✕

SPEED

SLOW

CURRENT MISSION

NOT PRACTICAL

This is important because the learner should understand:

A security solution can be secure yet operationally unsuitable.

That's a very useful engineering lesson.

We shouldn't say physical transfer is bad

It could be highly secure in the right context.

Our conclusion is:

SECURE

BUT

NOT SUITABLE FOR THIS REMOTE TASK

That's more accurate.

12. ACT F — Select the Approved Key-Transfer Method

Scene 14 — Controlled Transfer Method Approved

The learner is now shown a Phantom Q controlled/trusted key-transfer option suitable for this exercise.

This should be deliberately abstract at Release 1.

Something like:

PHANTOM Q CONTROLLED KEY DELIVERY

Recipient verification required

Authorisation required

Secret release controlled

The learner evaluates:

NORMAL CHANNEL

✗ Secret exposed

PHYSICAL DELIVERY

✓ Secure

✗ Impractical for current mission

CONTROLLED METHOD

✓ Suitable

✓ Remote-compatible

✓ Requires Bob verification

Then selects:

CONTROLLED METHOD

Result:

KEY TRANSFER METHOD

APPROVED ✓

But:

KEY A7

NOT YET RELEASED

This is exactly where Incident 02 should end.

13. Why do we stop before Bob gets the key?

Because otherwise Incident 02 and Incident 03 become the same incident.

Incident 02 teaches:

How symmetric protection works and why key distribution is a security problem.

Incident 03 teaches:

How to complete the trusted operational relationship before releasing the shared secret.

That gives each incident a clear purpose.

14. Player actions throughout Incident 02

The learner should physically:

1. enter the Cryptography Lab;
2. open `03_ACCESS_SCHEDULE.XLSX`;
3. inspect readable content;
4. attempt normal transfer;
5. receive the protection requirement;
6. create Key A7;
7. assemble plaintext + A7 $\rightarrow$ encrypt;
8. run encryption;
9. inspect ciphertext;
10. send ciphertext to Bob;
11. see Bob receive the package;
12. watch Bob fail to open it;
13. compare Alice/Bob key state;
14. determine Bob requires A7;
15. attempt normal key sending;
16. see why the secret becomes exposed;
17. inspect physical-token delivery;
18. identify it as secure but impractical;
19. compare the available delivery methods;
20. approve the controlled key-transfer method.

That is a substantial role-play incident without unnecessary maths.

15. Information flow

This must also stay tightly controlled.

Beginning

The learner knows:

Readable information cannot simply be trusted
to travel through the existing process.

They do **not** know how to solve it.

After plaintext inspection

Readable information = plaintext.

After encryption

Plaintext can be transformed
into ciphertext using Key A7.

After Bob receives it

The protected information can reach Bob.

After Bob fails to read it

Bob needs A7.

After normal key-transfer simulation

A shared secret cannot simply be exposed
through the same communication route.

After physical delivery

Security alone is not enough;
the method must also work operationally.

End

A controlled method exists,
but Bob must be verified before A7 is released.

That is the exact knowledge boundary.

16. Character roles

Trainee

The trainee moves from:

“We need to protect the file.”

to:

“Protecting the file creates a key-management responsibility.”

That is a meaningful progression.

Alice

Alice is still:

Origin / Protector

She owns:

03_ACCESS_SCHEDULE.XLSX
KEY A7

at this point.

She should not become a crypto lecturer.

Her practical concern is:

“Bob needs the information, but we shouldn't expose it.”

Bob

Bob remains remote.

His progression:

```
NO FILE
not the issue anymore

↓

CIPHERTEXT RECEIVED ✓

↓

CANNOT READ ✕

↓

NEEDS A7
```

He is the reason the learner experiences the decryption/key problem.

Eve

Eve can support:

- security-risk testing;
- evaluating the channel;
- explaining why sending a secret through an exposed route is problematic.

She does **not**:

- steal A7;
 - intercept current live traffic;
 - become the adversary;
 - cause the key problem.
-

17. World behaviour

The Cryptography Lab should visibly evolve during the incident.

At entry:

Unlocked objects:

PLAINTEXT
KEY
ENCRYPT
CIPHERTEXT

Later:

DECRYPT
SHARED KEY STATE
KEY DELIVERY OPTIONS

But things such as:

PUBLIC KEY
PRIVATE KEY
DIGITAL SIGNATURE

remain locked.

The learner may physically see those modules as unavailable, creating future curiosity without introducing them yet.

18. Core gameplay mechanics

Incident 02 should rely primarily on **object manipulation**.

Not reading-heavy screens.

Examples:

Encryption assembly

```
[ PLAINTEXT ] + [ KEY A7 ]  
      ↓  
[ ENCRYPT ]  
      ↓  
[ CIPHERTEXT ]
```

Key state comparison

ALICE

A7 ✓

BOB

A7 ✗

Delivery-route choice

NORMAL CHANNEL

PHYSICAL TOKEN

CONTROLLED METHOD

The learner sees consequences from each choice.

19. Success/failure behaviour

Again: no generic “wrong”.

Send plaintext

TRANSFER BLOCKED

PQ-001 protection requirement
has not been satisfied.

Encrypt without A7

PROCESS INCOMPLETE

Encryption requires key material.

Give Bob the wrong key

We should probably defer actual wrong-key testing to Incident 03, where we explicitly use B4.

So Incident 02 need not introduce that yet.

Send A7 through normal channel

Rather than hard block immediately:

RISK DETECTED

The shared secret would travel
through the same communication path.

Then perhaps the learner can run the simulation and see:

SECRET EXPOSURE
POSSIBLE

Choose physical delivery

Not wrong.

The result should say:

SECURITY
ACCEPTABLE

CURRENT REMOTE TASK
NOT PRACTICAL

This teaches trade-offs.

Select controlled method before reviewing alternatives

We could permit it, but award weaker understanding/proficiency.

Better gameplay would encourage the learner to test the available methods first.

20. Real concepts underneath the gameplay

Player experience	Concept
Opens readable File 3	Plaintext
Normal transfer blocked	Need for protection
Creates A7	Shared secret
Plaintext + A7	Symmetric encryption

Player experience	Concept
Ciphertext appears	Protected representation
Bob receives but cannot read	Decryption dependency
Bob lacks A7	Same-key relationship
Sends A7 normally	Key-distribution risk
Physical token	Out-of-band/physical delivery concept
Secure but slow	Security vs operational usability
Controlled transfer method	Key-management / controlled release

So the learner leaves knowing much more than simply “encryption exists”.

21. What Incident 02 deliberately does NOT teach

Do not introduce yet:

- public/private keys;
- RSA;
- digital signatures;
- IP addresses;
- network tracing;
- certificates;
- S1;
- attackers intercepting the key;
- QKD;
- quantum threat.

Also do not tell the learner:

“Symmetric encryption is insecure.”

That would be false and would damage the later architecture.

The correct lesson is:

Symmetric protection works, but the shared secret must be managed and delivered safely.

22. Persistent outputs created

Incident 02 adds several new permanent objects to the campaign.

```
KEY A7
```

This is especially important because it survives through:

```
Incident 02
↓
Incident 03
↓
Incident 04
↓
RETIRED
```

The incident also creates:

```
Protected 03_ACCESS_SCHEDULE package
Encryption workflow
Key-distribution assessment
Approved controlled delivery method
```

So the campaign evidence/history becomes:

```
PQ-001
|
├─ Prologue evidence
├─ Incident 01 assessment
|
└─ Incident 02 protection response
    ├─ plaintext source
    ├─ Key A7
    ├─ ciphertext
    ├─ Bob receipt
    ├─ key-state mismatch
    └─ approved key-delivery method
```

23. Incident 02 success condition

The learner succeeds when they can establish:

```
03_ACCESS_SCHEDULE.XLSX
PLAINTEXT IDENTIFIED ✓
```

KEY A7

CREATED ✓

ENCRYPTION

SUCCESSFUL ✓

CIPHERTEXT

DELIVERED TO BOB ✓

BOB CAN READ

NO

REASON

BOB DOES NOT HAVE A7 ✓

NORMAL KEY DELIVERY

UNSUITABLE ✓

PHYSICAL DELIVERY

SECURE BUT IMPRACTICAL ✓

CONTROLLED METHOD

APPROVED ✓

KEY RELEASE

PENDING

Notice that:

BOB STILL CANNOT READ THE FILE.

That is intentional.

24. End-state role-play

At the end of Incident 02, Bob's workstation still displays:

PROTECTED FILE

RECEIVED ✓

KEY A7

REQUIRED

Alice's side:

KEY A7

AVAILABLE ✓

The learner's controlled-transfer interface says:

```
METHOD
APPROVED ✓

NEXT REQUIREMENT
VERIFY RECIPIENT
```

Then the new operational objective appears:

COMPLETE THE SYMMETRIC LINK

That becomes Incident 03.

25. Exact handoff into Incident 03

The transition should be:

```
INCIDENT 02

We successfully protected the information.
    ↓
Bob received ciphertext.
    ↓
Bob cannot recover the information.
    ↓
He needs the same shared secret.
    ↓
A controlled key-transfer method exists.
    ↓
But before releasing A7:

WHO IS BOB?
IS HE AUTHORISED?
ARE WE RELEASING THE SECRET
TO THE CORRECT RECIPIENT?
    ↓
INCIDENT 03
COMPLETE THE SYMMETRIC LINK
```

That is a clean bridge.

Incident 02 master flow

```
INCIDENT 01 COMPLETE
↓
```


This keeps Incident 02 focused: **the learner builds symmetric protection, proves that it works, then discovers the operational problem of distributing the shared secret.** It does not prematurely solve recipient verification or key release.